

EJERCICIOS METAPROGRAMACIÓN

1. Desarrollar un sistema de logging que permita registrar información de cualquier objeto en tiempo de ejecución. Se pide:
 - a. Definir una jerarquía de clases para representar las piezas de ajedrez. Definir sus constructores (con y sin parámetros), métodos set/get, método imprimir que muestre la información de la Pieza y método que muestre una descripción del movimiento de las mismas.
 - b. Crear la clase **IntrospectionClass** que contenga un método **informaciónClase(Object obj)** que reciba por parámetro un objeto tipo Object y aplique **Introspection** para imprimir por consola:
 - i. El nombre de la clase, el cual se puede obtener de dos formas:
A partir de un objeto:

```
Object cadena = "Hola";  
Class<?> claseCadena = cadena.getClass();
```

A partir del nombre de la clase:

```
Class<?> clasePersona = Class.forName("paquete.Persona");
```



```
Class<?> clase = objeto.getClass()  
System.out.println("Clase: " + clase.getName());
```
 - ii. Los atributos de la clase, modificador de acceso, el tipo de dato y su respectivo estado. Investigar los métodos **getModifiers()**, **getDeclaredFields** y **getFields()**. En el caso de modificadores privados, habilitar el acceso a campos privados: `campo.setAccessible(true)` para anular el control de acceso. Para obtener el valor del campo: `Object valor = campo.get(objeto);`
 - iii. Los constructores de la clase y parámetros que reciben. Investigar los métodos **getConstructors()** y **getDeclaredConstructors()**.
 - iv. Los métodos de la clase, sus modificadores de acceso, el tipo que retorna y parámetros que recibe. Investigar los métodos **getMethods()**, **getDeclaredMethods()**, **getReturnType()**, **getParameterTypes()**.
 - v. Obtener la superclase de las clases hijas. Investigar el método **getSuperclass()**.
 - vi. Definir interfaces para representar el movimiento en diagonal del Alfil y la Reina. Verificar si las clases implementan dicha interface utilizando **getInterfaces()**.
2. A partir de la jerarquía de clases planteada en el ejercicio N° 1, aplicar **Intercession** para incorporar movimientos especiales de algunas piezas como:
 - a. **Enroque:** Verificar las condiciones para el enroque antes de permitir el movimiento del rey y la torre.
 - b. **Promoción de peones:** Interceptar la llamada cuando un peón llega a la última fila y ofrecer opciones de promoción.

3. Aplicar **Self-modification** para crear instancias de todas las piezas e invocar **movimiento()** para mostrar el movimiento de cada una de ellas.
4. Implementa un generador de clases genéricas en Java que permita crear la clase genérica que corresponde a una clase dada. Para ello se debe analizar la clase dada, obtener sus atributos, constructores y métodos (con todo su detalle asociado).

```
public class Par {  
    private String cad1;  
    private String cad2;  
  
    public Par (String a, String b){ cad1 = a;  
    cad2 = b;  
    }  
  
    private void SetCad1(String c){  
        cad1=c;  
    }  
  
    private void SetCad2(String c){  
        cad2=c;  
    }  
  
    public String getCad1(){  
        return cad1;  
    }  
  
    public String getCad2(){  
        return cad2;  
    }  
}
```